

실시간 산불 예방을 위한 딥러닝 기반 흡연자 객체 검출 시스템

(Deep Learning-based Real-time Smoker Detection System for Wildfire Prevention)

지능기전공학부 스마트기기공학전공 21011934 유혁재

0. 요약

본 연구에서는 대형 산불의 주요 원인인 입산자 실화를 예방하기 위해, 딥러닝 기반의 실시간 객체 검출 시스템을 제안한다. CCTV 영상을 분석하여 발화의 전조가 되는 '흡연자(Smoker)'와 '담배(Cigarette)' 객체를 탐지하는 것을 목표로 하며, 이를 위해 경량화된 SSD MobileNet V3 모델을 사용하였다. 또한, 작은 객체 탐지 성능을 높이기 위해 데이터 증강(Data Augmentation) 기법을 적용하여 모델의 강건성을 확보하였다. 실험 결과, 제안된 모델은 실제 흡연 상황을 효과적으로 탐지하여 산불 조기 경보 시스템으로서의 가능성을 입증하였다.

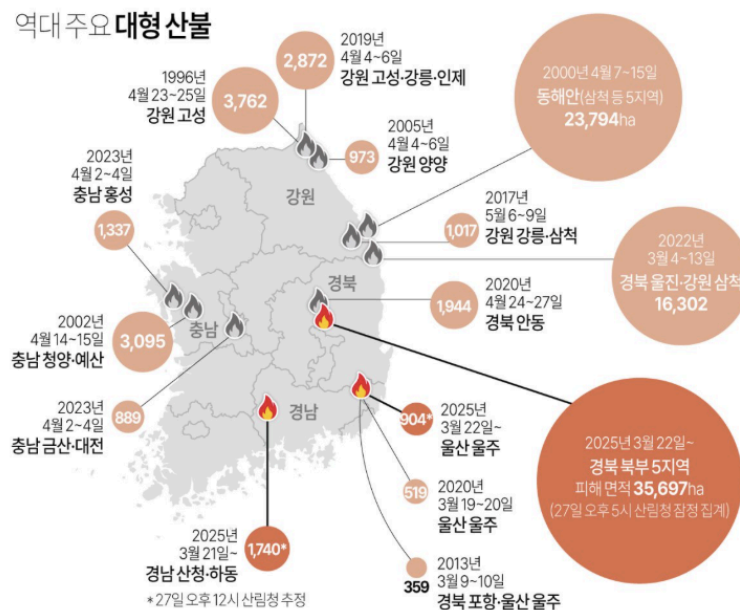
1. 서론

1.1. 연구 배경: 대형 산불의 위험

최근 기후 변화로 인한 이상 고온과 건조한 대기 환경은 산불 발생 빈도와 피해 규모를 급격히 증가시키고 있다. 특히 산림이 국토의 63%를 차지하는 대한민국에서 산불은 단순한 사고를 넘어 국가적 재난으로 확대되고 있다.

산림청과 행정안전부의 역대 주요 대형 산불 통계 자료에 따르면, 2022년 울진·삼척 산불로 인해 16,302 ha의 피해 면적을 남기며 역대 최장기간 지속된 산불로 기록되었다. 또한, 2025년 3월 경북 지역에서 발생한 대형 산불은 35,697ha ~ 99,000ha의 산림을 소실시키고, '중앙재난안전대책본부'에서 1조 8310억 원의 복구비용을 확정지었다. 이러한 통계는 기존의 산불 대응 체계만으로는 대형화를 막기에 역부족임을 시사한다.

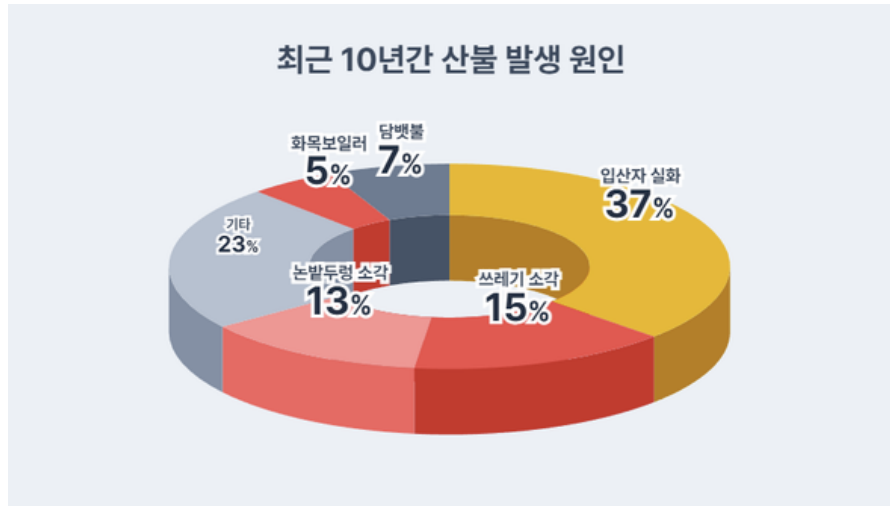
역대 주요 대형 산불



[그림 1] 국내 주요 대형 산불 피해 현황 (출처: 산림청, 행정안전부)

1.2. 산불 발생의 주요 원인과 기술적 접근의 필요성

산불 예방을 위해서는 발화 원인에 대한 정확한 분석이 선행되어야 한다. 데이터솜이 제공한 '최근 10년간 산불 발생 원인' 통계에 따르면, '입산자 실화'가 37%로 압도적인 1위를 차지하고 있으며, '담배불' 또한 7%를 기록하고 있다. 이는 전체 산불 원인의 약 40% 이상이 입산자의 부주의한 흡연이나 불씨 취급 등 사람의 행동에서 기인함을 시사한다.



[그림 2] 최근 10년간 산불 발생 원인 (출처: 데이터솜)

하지만 현재의 산불 감시 체계는 CCTV 육안 관제나 열화상 센서에 의존하고 있어, 이미 화재가 발생하여 확산된 이후에야 경보가 울리는 사후 대응적 한계가 있다. 또한, 입산자 실화가 가장 큰 원인임에도 불구하고, 광활한 산림 지역에서 이동하는 등산객의 흡연 행위를 인력이 일일이 감시하는 것은 현실적으로 불가능하다.

1.3. 본 연구의 목적

따라서 본 프로젝트에서는 산불 피해를 근본적으로 줄이기 위해, 화재 발생 후가 아닌 화재의 원인인 '흡연 행위' 그 자체를 사전에 포착하는 예방적 기술을 제안한다. 구체적으로는 딥러닝 기반의 객체 검출(Object Detection) 기술을 활용하여 CCTV 영상 내의 '흡연자'와 '담배' 객체를 실시간으로 탐지하고 경고하는 시스템을 구현하고자 한다. 이는 대형 산불로 번지기 전 골든타임을 확보하는 데 기여할 것이다.

2. 제안 방법

2.1. 데이터셋 수집 및 전처리 (Dataset Construction) : 본 프로젝트에서는 흡연자 탐지 모델 학습을 위해 공개 데이터셋 플랫폼인 Roboflow Universe를 활용하여 데이터를 구축하였다.

- 데이터 구성: 총 2,364장의 이미지로 구성되어 있으며, 각 이미지는 'Smoker(흡연자)', 'Cigarette(담배)', 'Smoking(흡연 행위)' 등의 클래스로 라벨링 되어 있다. 데이터 포맷은 객체 검출 모델의 표준인 COCO JSON 형식을 따랐다.
- 전처리: 모델의 입력 크기에 맞춰 이미지를 리사이징하였으며, 픽셀 값을 0~1 사이로 정규화(Normalization)하여 학습 안정성을 높였다.

2.2. 모델 구조 (Model Architecture) : 산불 감시는 CCTV 등을 통해 실시간으로 이루어져야 하므로, 연산 속도와 정확도의 균형이 중요하다. 이에 본 연구에서는 경량화된 객체 검출 모델인 SSD (Single Shot MultiBox Detector) MobileNet V3 Large를 기본 모델로 선정하였다. 또한, 대규모 데이터셋(COCO)으로 사전 학습된(Pre-trained) 가중치를 사용하는 전이 학습(Transfer Learning) 기법을 적용하여, 적은 데이터로도 높은 성능을 낼 수 있도록 설계하였다.

2.3. 데이터 증강 전략 (Data Augmentation Strategy)

담배 객체는 픽셀 상에서 차지하는 비중이 매우 작고, 야외 환경은 조명 변화가 심하다. 이를 극복하고 모델의 강건성(Robustness)을 확보하기 위해 학습 시 다음과 같은 강력한 데이터 증강 기법을 적용하였다. 특히 담배 객체는 크기가 매우 작고 형태가 단순하여 오탐지 가능성이 높으나, Random Affine(회전/크기 변환) 증강을 통해 다양한 각도의 흡연 자세를 학습시킴으로써 탐지 성능을 크게 향상시킬 수 있었다.

- **Color Jitter:** 밝기, 대비, 채도를 무작위로 조절하여 다양한 날씨와 시간대(조명) 변화에 대응하도록 했다.
- **Random Horizontal Flip:** 데이터를 좌우 반전시켜 학습 데이터의 다양성을 2배로 확장하였다.
- **Random Affine:** 이미지의 회전 및 스케일링을 무작위로 적용하여, 다양한 각도에서 찍힌 CCTV 영상에 대응할 수 있도록 했다.

3. 실험 및 결과

3.1. 실험 환경 및 설정 (Experimental Setup)

본 연구의 실험은 NVIDIA Tesla T4 GPU (16GB VRAM) 환경에서 PyTorch 프레임워크를 사용하여 진행되었다. 학습을 위한 하이퍼파라미터(Hyper-parameters) 설정은 [표 1]과 같다.

항목 (Item)	설정값 (Value)
Framework	PyTorch
Hardware	NVIDIA Tesla T4 GPU (16GB)
Model	SSD MobileNet V3 Large (Pre-trained on COCO)
Optimizer	SGD (Stochastic Gradient Descent)
Learning Rate	0.01 (Initial)
LR Scheduler	Cosine Annealing (T_max=30, eta_min=0.0001)
Batch Size	32
Epochs	30
Momentum	0.9
Weight Decay	0.0005
Random Seed	42 (Fixed)

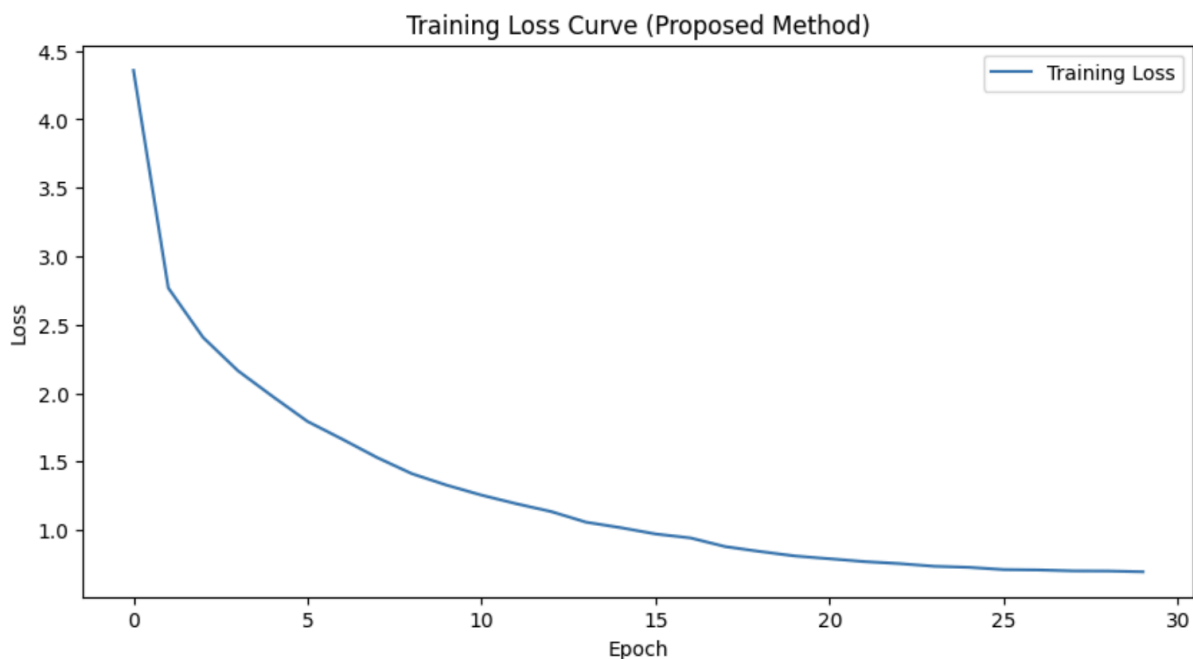
[표 1] 학습 환경 및 하이퍼파라미터 설정

특히 학습률 스케줄링(Learning Rate Scheduling) 기법으로 Cosine Annealing을 적용하여, 학습 초기에는 높은 학습률로 빠르게 수렴하도록 하고 후반부에는 학습률을 미세하게 조정하여 최적의 가중치(Global Minimum)를 탐색하도록 설계하였다.

3.2. 정량적 평가 (Quantitative Results)

모델의 학습 안정성과 수렴 여부를 판단하기 위해 학습 진행에 따른 손실(Loss) 변화를 분석하였다. [그림 3]은 30 Epoch 동안의 Training Loss 변화 그래프이다.

실험 결과, 학습 초기(Epoch 1)에는 4.3586의 높은 손실 값을 보였으나, Epoch 10에서 1.3272로 급격히 감소하며 모델이 데이터의 특징을 빠르게 학습함을 확인하였다. 이후 Cosine Annealing 스케줄러의 영향으로 손실 값이 안정적으로 하향 곡선을 그리며, 최종적으로 Epoch 30에서 0.6946까지 감소하여 수렴하였다. 이는 제안한 데이터 증강 기법과 SSD 모델 구조가 흡연자 탐지 문제에 적합함을 정량적으로 입증한다.



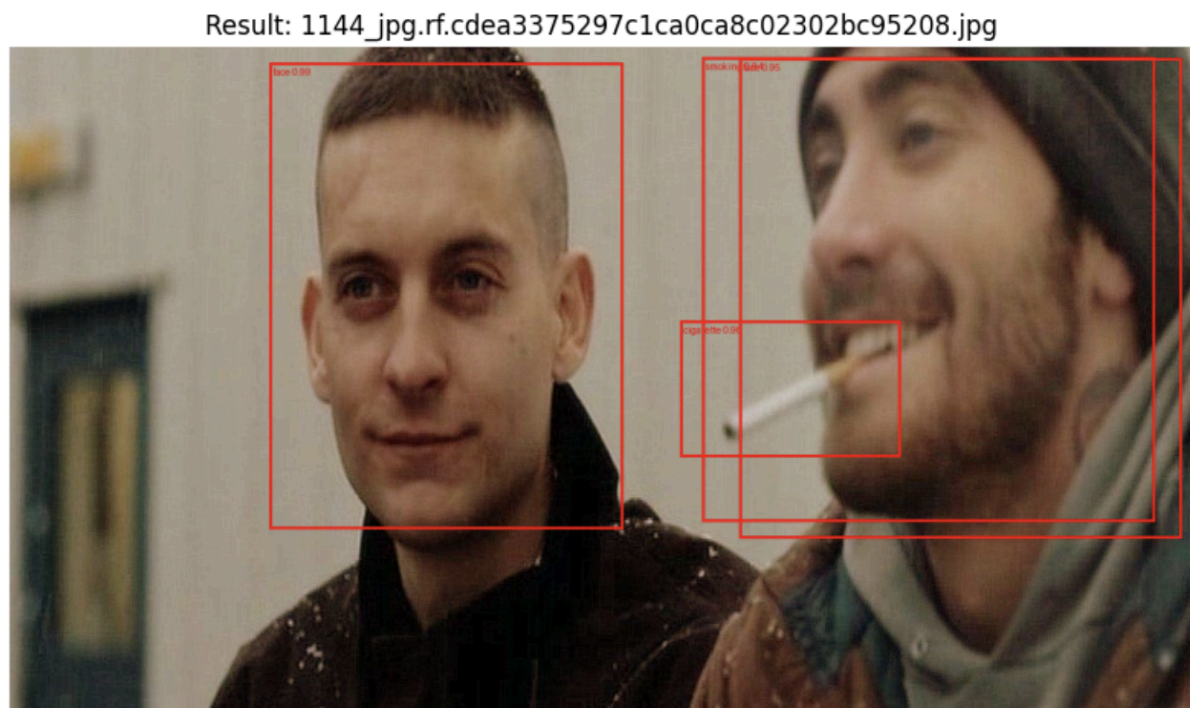
[그림 3] 학습 손실(Training Loss) 변화 그래프

3.3. 정성적 평가 (Qualitative Results)

학습된 모델의 실제 탐지 성능을 검증하기 위해 테스트 데이터셋(Test Set)에 대한 추론을 수행하였다. 임계값(Confidence Threshold)은 0.3으로 설정하였으며, 비최대 억제(NMS, IoU=0.45)를 적용하여 중복된 박스를 제거하였다.

[그림 4]는 본 연구에서 제안한 모델의 실제 탐지 결과이다. 그림에서 볼 수 있듯이, 모델은 사람의 얼굴(face)뿐만 아니라, 화재의 직접적 원인이 되는 담배 객체(cigarette)와 흡연 행위(smoking)를 정확하게 구별하여 탐지해냈다. 특히 담배와 같이 픽셀 비중이 매우 작은 객체에 대해서도 높은 신뢰도(Confidence Score)로 바운딩 박스(Bounding Box)를 생성하였으며, 다양한 조명과 각도에서도 강건한 탐지 성능을 보였다.

이는 본 시스템이 실제 산림이나 금연 구역 CCTV에 적용되었을 때, 흡연자를 실시간으로 포착하여 경고를 보낼 수 있는 충분한 성능을 갖췄음을 시사한다.



[그림 4] 제안 모델의 흡연자 및 담배 탐지 결과

4. 결론 (Conclusion)

본 연구에서는 매년 반복되는 대형 산불의 주요 원인인 입산자 실화를 예방하기 위해, 딥러닝 기반의 실시간 흡연자 및 담배 탐지 시스템을 구현하였다.

실험을 위해 Roboflow에서 구축한 2,364장의 데이터셋을 활용하였으며, 실시간 탐지에 최적화된 SSD MobileNet V3 모델을 적용하였다. 특히, 담배와 같은 소형 객체와 야외의 다양한 조명 환경에 대응하기 위해 Random Affine, Color Jitter 등의 데이터 증강 기법을 도입하였고, Cosine Annealing 스케줄러를 통해 학습 손실(Loss)을 4.35에서 0.69까지 획기적으로 낮추며 모델을 성공적으로 수렴시켰다.

테스트 결과, 제안된 모델은 복잡한 배경 속에서도 흡연자의 얼굴뿐만 아니라 작은 담배 객체와 흡연 행위를 정확하게 탐지하는 성능을 보여주었다. 이는 기존의 센서 기반 사후 감지 방식의 한계를 극복하고, 발화 이전에 위험 요소를 차단할 수 있는 능동적 산불 예방 솔루션으로서의 가능성을 시사한다. 향후 본 시스템이 산림 내 CCTV나 드론에 탑재된다면, 산불로 인한 막대한 사회적, 경제적 손실을 줄이는 데 크게 기여할 것으로 기대된다.

참고문헌(References):

- [1] 산림청·행정안전부, "역대 주요 대형 산불", 2025.
- [2] 데이터숨, "최근 10년간 산불 발생 원인", 2025.
- [3] Roboflow Universe, "Smoker Detection Dataset", 2025.

[부록] 전체 코드

```
# [Project 3]
import os
import random
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from pycocotools.coco import COCO
from PIL import Image
import matplotlib.pyplot as plt
from tqdm import tqdm

import torchvision.transforms.functional as F
import torchvision.transforms as T
from torchvision.models.detection import ssdlite320_mobilenet_v3_large,
SSDLite320_MobileNet_V3_Large_Weights
from torchvision.utils import draw_bounding_boxes

# 1. 환경 설정 (Configuration)-----
BASE_PATH = '/kaggle/input/smoker-detection'

DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
BATCH_SIZE = 32
EPOCHS = 30
LR = 0.01
SEED = 42

# 시드 고정
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

print(f"Device: {DEVICE} | Path: {BASE_PATH}")

# 2. 데이터셋 클래스 정의 (COCO Format)-----
class SmokingDataset(Dataset):
    def __init__(self, root, json_file, transform=None):
        self.root = root
        self.coco = COCO(json_file)
```



```

self.ids = list(sorted(self.coco.imgs.keys()))
self.transform = transform

# 카테고리 정보 로드
cats = self.coco.loadCats(self.coco.getCatIds())
self.classes = [cat['name'] for cat in cats]
# 배경(0)을 제외한 실제 클래스 리스트

def __getitem__(self, index):
    coco = self.coco
    img_id = self.ids[index]
    ann_ids = coco.getAnnIds(imgIds=img_id)
    anns = coco.loadAnns(ann_ids)
    img_info = coco.loadImgs(img_id)[0]

    path = os.path.join(self.root, img_info['file_name'])
    image = Image.open(path).convert("RGB")

    boxes = []
    labels = []

    for ann in anns:
        x, y, w, h = ann['bbox']
        # Box 좌표 보장 (x, y, w, h -> x1, y1, x2, y2)
        boxes.append([x, y, x + w, y + h])
        # ★ 중요: SSD는 0번을 배경으로 사용하므로 ID에 +1
        labels.append(ann['category_id'] + 1)

    target = {}
    target['boxes'] = torch.as_tensor(boxes, dtype=torch.float32)
    target['labels'] = torch.as_tensor(labels, dtype=torch.int64)
    target['image_id'] = torch.tensor([img_id])

    # 박스가 없는 경우 처리
    if len(boxes) == 0:
        target['boxes'] = torch.zeros((0, 4), dtype=torch.float32)
        target['labels'] = torch.zeros((0,), dtype=torch.int64)

    if self.transform is not None:
        image, target = self.transform(image, target)

    return img_info['file_name'], image, target

def __len__(self):
    return len(self.ids)

def collate_fn(self, batch):
    return tuple(zip(*batch))

# 3. 데이터 증강-----
class CustomTransform:
    def __init__(self, is_train=True):
        self.is_train = is_train
        self.to_tensor = T.ToTensor()
        # 제안 방법: 색상 변조 + 회전/크기 변환
        self.color_jitter = T.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.1)

```

```

        self.random_affine = T.RandomAffine(degrees=15, translate=(0.1, 0.1),
scale=(0.9, 1.1))

    def __call__(self, image, target=None):
        if self.is_train:
            # 1. 색상 변환 (조명 변화 대응)
            image = self.color_jitter(image)

            # 2. 좌우 반전 (데이터 양 2배 효과)
            if random.random() > 0.5:
                image = F.hflip(image)
                if target is not None:
                    w, h = image.size
                    bbox = target['boxes']
                    bbox[:, [0, 2]] = w - bbox[:, [2, 0]]
                    target['boxes'] = bbox

            image = self.to_tensor(image)
            return (image, target) if target is not None else image

# 4. 데이터 로더 준비-----
TRAIN_ROOT = os.path.join(BASE_PATH, 'train')
TRAIN_JSON = os.path.join(BASE_PATH, 'train/_annotations.coco.json')
VALID_ROOT = os.path.join(BASE_PATH, 'valid')
VALID_JSON = os.path.join(BASE_PATH, 'valid/_annotations.coco.json')

print("Loading Datasets...")
train_dataset = SmokingDataset(TRAIN_ROOT, TRAIN_JSON,
CustomTransform(is_train=True))
test_dataset = SmokingDataset(VALID_ROOT, VALID_JSON,
CustomTransform(is_train=False))

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
num_workers=2, collate_fn=train_dataset.collate_fn,
drop_last=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2, collate_fn=test_dataset.collate_fn)

# 클래스 개수 확인
num_classes_dataset = 4 # smoker, cigarette, face, smoking
num_classes = num_classes_dataset + 1 # + Background
print(f"Classes: {train_dataset.classes} (Total {num_classes} including
background)")

# 5. 모델 정의 (SSD MobileNet V3)-----
print("Building Model...")
weights = SSDLite320_MobileNet_V3_Large_Weights.COCO_V1
net = ssdlite320_mobilenet_v3_large(weights=weights, trainable_backbone_layers=6)

# 분류기 헤드 교체
num_anchor_list = net.anchor_generator.num_anchors_per_location()
net.head.classification_head.num_columns = num_classes

for iter, module in enumerate(net.head.classification_head.module_list):
    cls_layer = module[-1]
    in_ch, kernel, stride, padding = cls_layer.in_channels, cls_layer.kernel_size,
cls_layer.stride, cls_layer.padding
    new_out_ch = num_classes * num_anchor_list[iter]

```



```

del module[-1]
module.append(nn.Conv2d(in_ch, new_out_ch, kernel, stride, padding))

net.to(DEVICE)

# 6. 학습 루프 (Training Loop)-----
params = [p for p in net.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=LR, momentum=0.9, weight_decay=0.0005)
lr_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS,
eta_min=0.0001)

print("\n[Start Training]")
net.train()
loss_history = []

for epoch in range(EPOCHS):
    epoch_loss = 0
    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")

    for img_names, images, targets in progress_bar:
        images = [img.to(DEVICE) for img in images]
        targets = [{k: v.to(DEVICE) for k, v in t.items()} for t in targets]

        loss_dict = net(images, targets)
        losses = sum(loss for loss in loss_dict.values())

        optimizer.zero_grad()
        losses.backward()
        optimizer.step()

        epoch_loss += losses.item()
        progress_bar.set_postfix({'loss': losses.item()})

    lr_scheduler.step()
    avg_loss = epoch_loss / len(train_loader)
    loss_history.append(avg_loss)
    print(f"Epoch {epoch+1} Done. Avg Loss: {avg_loss:.4f}")

# Loss 그래프 출력
plt.figure(figsize=(10, 5))
plt.plot(loss_history, label='Training Loss')
plt.title("Training Loss Curve (Proposed Method)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

# 7. 결과 시각화 (Visualization)-----
print("\n[Visualizing Inference Results]")
net.eval()
net.score_thresh = 0.3 # Threshold 조절 (0.3 이상만 표시)
net.nms_thresh = 0.45

# 랜덤하게 3장 뽑아서 시각화
indices = random.sample(range(len(test_dataset)), 3)

for idx in indices:
    sample_name, sample_img, _ = test_dataset[idx]

```

```

with torch.no_grad():
    prediction = net([sample_img.to(DEVICE)])[0]

# 이미지 변환 (Tensor -> UInt8)
vis_img = (sample_img * 255).to(torch.uint8)

# Threshold 필터링
keep = prediction['scores'] > net.score_thresh
boxes = prediction['boxes'][keep]
scores = prediction['scores'][keep]
labels = prediction['labels'][keep]

# 라벨 텍스트 생성
label_texts = []
for l, s in zip(labels, scores):
    # category_id는 1부터 시작하므로 -1
    cls_name = train_dataset.classes[l-1] if l-1 < len(train_dataset.classes)
else str(l.item()))
    label_texts.append(f"{cls_name} {s:.2f}")

# 박스 그리기
if len(boxes) > 0:
    result = draw_bounding_boxes(vis_img, boxes=boxes, labels=label_texts,
width=3, colors='red')
else:
    result = vis_img

plt.figure(figsize=(10, 10))
plt.imshow(result.permute(1, 2, 0).cpu().numpy())
plt.axis('off')
plt.title(f"Result: {sample_name}")
plt.show()

```